

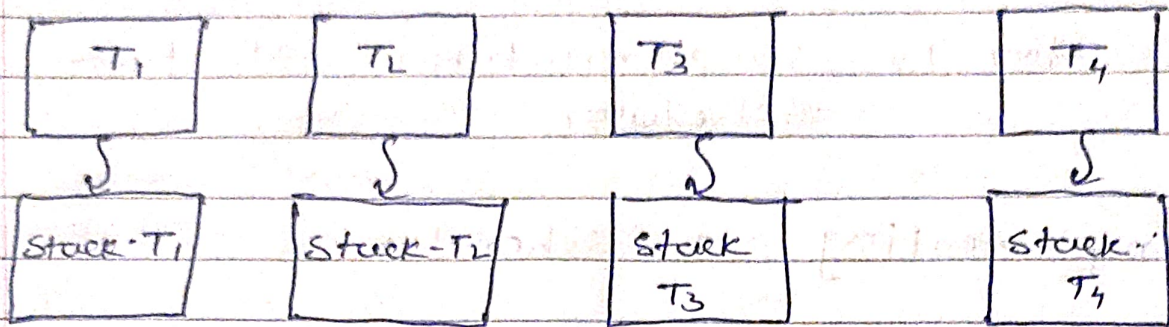
* Section 16: Implementation of task Scheduler

• Implementing a scheduler.

- Lets implement a scheduler which schedule a multiple user tasks in round-robin fashion by carrying out the context switch operation.
- Round robin scheduling method is, time slices are assigned to each task in equal portion and in circular order.
- First will use systick handler to carry out the context switch operation between multiple tasks.
- Later will be we change the code using pendsv handler.

• User tasks

- We will be using 4 user tasks in this application

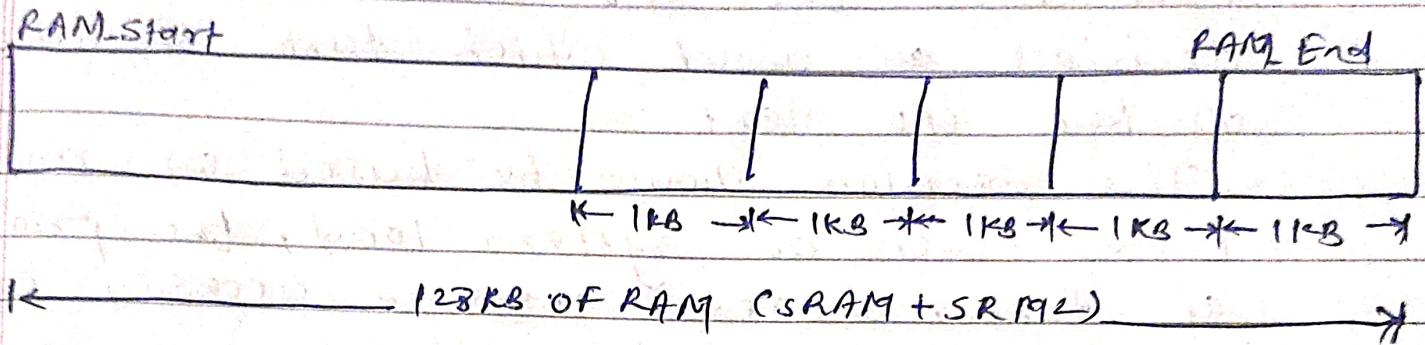


• What is a task?

- A task is nothing but a piece of code, or you can call it a 'c' function, which does a specific job when it allowed to run on CPU.
- A task has its own stack to create its local variables when it is run on the CPU. Also when scheduler decided to remove a task from CPU, scheduler first saves the context (state) of the task in task's private stack.
- So, in summary, a piece of code or a function is called a task when it is schedulable and never loses its 'state' unless it is deleted permanently.

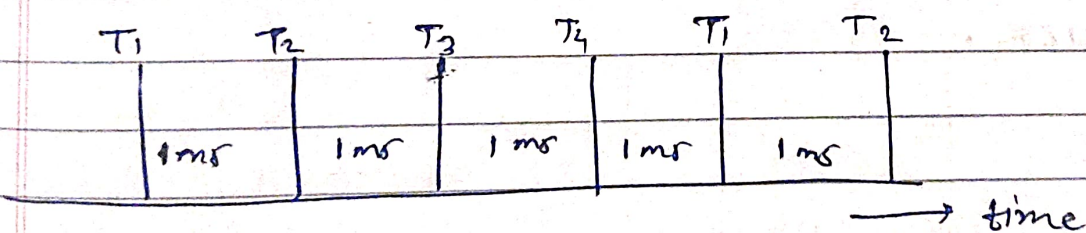
Good morning, my name is ~~seger~~ more.

• Stack Assessment



• Scheduling policy selection

- We will be using round robin pre-emptive scheduling
- No task priority
- We will use systick timer to generate a exception for every 1ms to run a scheduler code.



systick exception for every 1ms

• What is scheduling

— Scheduling is an algorithm which takes the decision of pre-empting a running task from the CPU and takes the decision about which task should run on the CPU next.

— The decision should be based on many factor such as system load, the priority of task, shared resource access, or a simple round robin method.

• What is context-switching

— Context switching is the procedure of switching out of the currently running task from the processor after ~~se~~ saving the task execution context or state and switching in the next task to run on the CPU by retrieving the past execution context or state of the task.

* case study for context switching

- case of T₁ switching out, T₂ switching in

Running T₁

Save the context of T₁ to T₁'s private stack

push

Context ~~push~~ saving

Save the PSP value of T₁

Get current value of PSP of T₂

Retrieve the context of T₂ from T₂'s private stack

Context Retrieving
pop

Run T₂

* Stack Frame

xPSR
Return Address (PC)
LR
R12
BB
R2
R1
R0

Symmetry: state of a stack

R0
R1
R2
⋮
R12

General-purpose registers

PSP

- stack pointer

LR (R14)
PC (R15)

Link Register
Program Counter

PSR

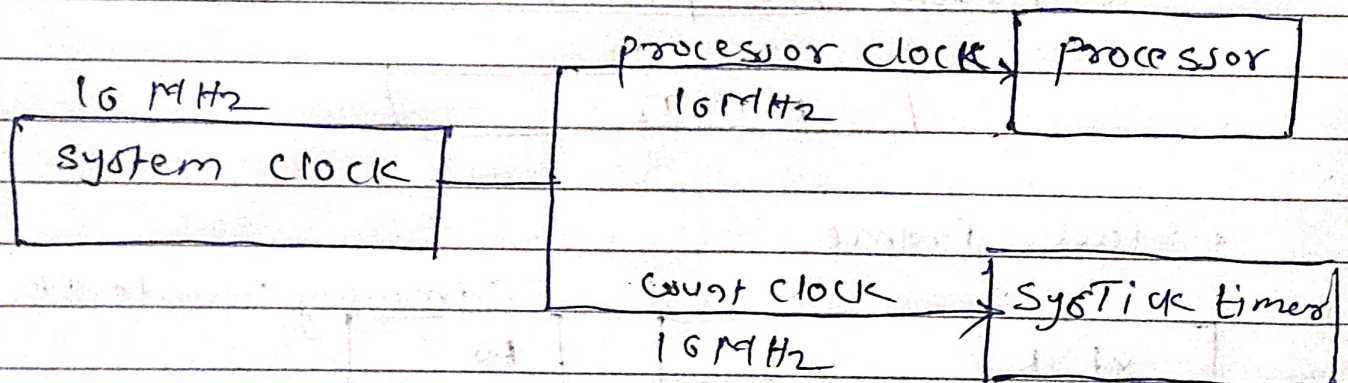
program status register

Task: configure the SysTick timer to produce exception for every 1ms.

• SysTick count value calculation

- processor clock = 16 MHz
- SysTick timer count clock = 16 MHz
- 1ms is 1 KHz in frequency domain
- So, to bring down SysTick timer count clock from 16 MHz to 1 KHz use a divider (reload value)
- Reload Value = 16000

• Clock distribution



SysTick timer count clock = 16 MHz

- For 1 count it takes $\frac{1}{16 \text{ MHz}}$ sec

i.e. 0.0625 μ sec delay $\rightarrow$ 1 count

So, 1 μ sec delay $\rightarrow$ 16 count

1 msec delay $\rightarrow$ 16000 count

Calculating RLOAD value!

- The RLOAD value is calculated according to its use
- For example: to generate a multi-shot timer with a period of N processor clock cycles, use a RLOAD value of $N-1$.

* Prologue and Epilogue

• function prologue:

- Code executed at the start of a function to:

Preserve caller context

Create a stack frame

Prepare space for local variable.

• function Epilogue

- Code executed before returning to:

Restore saved registers

Destroy stack frame

Return control to caller.